CST2572 Assessment 1: Medical Appointment and Information Website

This assessment is to build and test your skills in the deployment of a medical system for a local doctor's surgery. The web app should:

- Deploy a database system
- Interact and work with server-side technologies, fetching data which is drawn into the app's db
- Allow patients to book appointments with a specific doctor
- Allow doctors to store medical notes/treatments/medication details about a patient
- All aspects of both website configuration and the actual application should be GDPR compliant and secure

In addition:

- The website should be built utilising HTML, CSS and JavaScript
- Data should be stored securely in a correctly configured database
- A small case study of a described server and the hardening after a cyber-attack – full details on website

All submissions should include

- A spoken video showing the work and how it matches the mark scheme. This should only be 7 or so minutes long
- Code, text files, assets etc should all be neatly held in directories with appropriate readme.txt files
- All files submitted should be in one zip compressed format file i.e. not RAR or other.
- Videos should be made using Zoom and be clear to hear your vocal narrative. Failure to follow this will mean your work will be unmarked.

Information and checklist

Marking details:

Basic designs for both site and database (10%)

- Front page designs (8%)
- Database E/R diagram(2%)

IndexedDB database in use and managed (10%)

- Creation of DB, with JSON data from server (5%)
- Add (2%)
- Delete (1%)
- Update (2%)

Patient, Admin and Doctor account types and permissions (5%)

- Patient account type and permissions (1%)
- Admin account type and permissions (2%)
- Doctor account type and permissions (2%)

Admin functionality (Can manage patient's information name, address, DOB etc.) (10%)

- Can add details of patient (name, address, telephone, DOB, NHS number (5%)
- Can update/delete (5%)

Patient functionality (Can book appointments, look at medication instructions etc.) (10%)

- Can book appointments and request prescriptions (5%)
- Can see stored information and medical notes (5%)

Doctor functionality (Review patient history, medical notes, medicines) (10%)

- Can look at patient history, notes (5%)
- Can add to patient history, notes and medicines (prescriptions) (5%)

Validation checks and basic security (input sanitisation, password, email etc.) (5%)

- Accounts are password protected (2%)
- Inputs are sanitised, password reaches applied security standard, email input is valid (3%)

Security (Encryption) (10%)

- Encryption applied to passwords and other data (5%)
- Decryption and data in use after storage (5%)

Advanced (10%)

- JSON data collected from server and processed (5%)
- Encryption modules used from crypto lib or similar, must be AES 256 or better (5%)

Case Study / Report (20%)

- Servers / Protocols
- VMs
- Hacking detection / mitigation

Checklist:

Code included in zip ☐

Code is commented ☐

Code is modular (e.g. broken into functions) ☐

Readme.txt is supplied giving references, usage etc. ☐

Video included in zip ☐

Video time: ___ minutes

Video shows all claimed sections above ☐

Report (2000 words +/- 100) included in zip ☐

Report length: ___ words

The code, video, report and marksheet should be zipped and uploaded to the submission link.

I confirm all work done is my own, where third-party sources have been used, I have referenced them appropriately.

Signed:

Notes and Useful Information

This app is entirely client-side with no server required at all – it should run entirely in Google Chrome without any additions.

Example data can be pulled down into your database, which can be extended beyond the data given and updated by the user. No data needs to be written locally but a

dummy function can be available which would in-effect update a web database via a server. This does not have to be implemented.

You should deploy and test on Google Chrome browser, using pure JavaScript or compiled TypeScript

Google Chrome will not utilise Cookies locally (file: etc.)

Your code should run offline as well as online (PWA type app, assets locally stored and working preferably using service workers)

Large amounts of data should be stored in the indexedDB database

Web assets should be stored in the cache

You can use cookies or local storage for small bits of information but be aware of the Google Chrome limitation on cookies in the local filing system

You should *fetch* the JSON data stored on the server:

- https://jsethi-mdx.github.io/cst2572.github.io/doctors.json
- https://jsethi-mdx.github.io/cst2572.github.io/patients.json
- https://jsethi-mdx.github.io/cst2572.github.io/admin.json
- https://jsethi-mdx.github.io/cst2572.github.io/medicines.json
-

This data should be processed and placed in your indexeddb database